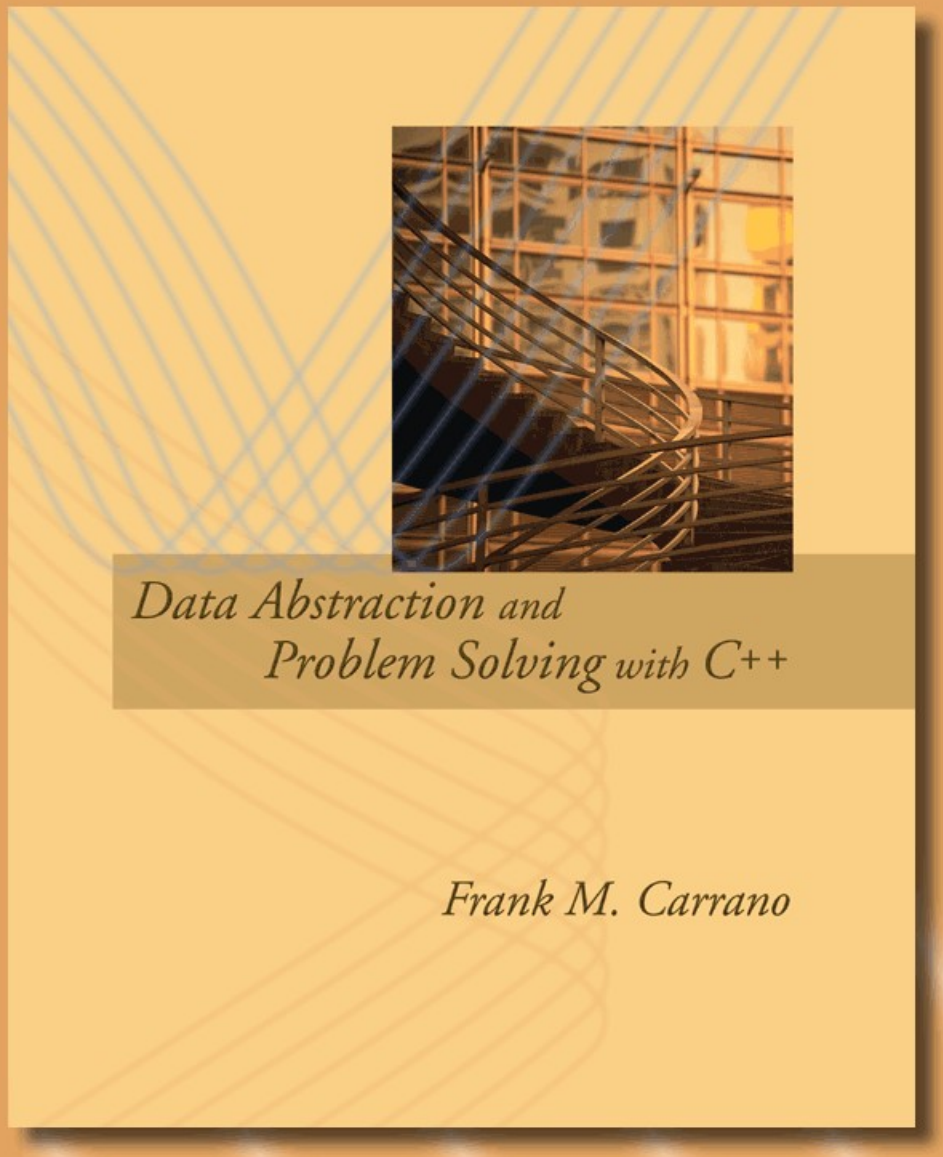
Chapter 7

Queues

Slides edited by Dr. Selim Aksoy



PEARSON
Addison
Wesley

© 2005 Pearson Addison-Wesley. All rights reserved.

The Abstract Data Type Queue

- A queue
  - New items enter at the back, or rear, of the queue
  - Items leave from the front of the queue
  - First-in, first-out (FIFO) property
    - The first item inserted into a queue is the first item to leave

© 2005 Pearson Addison-Wesley. All rights reserved

The Abstract Data Type Queue

- Queues
  - Are appropriate for many real-world situations
    - Example: A line to buy a movie ticket
  - Have applications in computer science
    - Example: A request to print a document
- A simulation
  - A study to see how to reduce the wait involved in an application

© 2005 Pearson Addison-Wesley. All rights reserved

The Abstract Data Type Queue

- ADT queue operations
  - Create an empty queue
  - Destroy a queue
  - Determine whether a queue is empty
  - Add a new item to the queue
  - Remove the item that was added earliest
  - Retrieve the item that was added earliest

© 2005 Pearson Addison-Wesley. All rights reserved

The Abstract Data Type Queue

```
bool isEmpty() const;

// Determines whether the queue is empty.

// Precondition: None.
// Postcondition: Returns true if the queue
// is empty; otherwise returns false.
```

© 2005 Pearson Addison-Wesley. All rights reserved

The Abstract Data Type Queue

```
bool enqueue(QueueItemType newItem);

// Inserts an item at the back of a queue.

// Precondition: newItem is the item to be
// inserted.
// Postcondition: If the insertion is
// successful, newItem is at the back of the
// queue.
```

© 2005 Pearson Addison-Wesley. All rights reserved

The Abstract Data Type Queue

```
bool dequeue();

// Dequeues the front of a queue.

// Precondition: None.
// Postcondition: If the queue is not empty,
// the item that was added to the queue
// earliest is deleted.
```

© 2005 Pearson Addison-Wesley. All rights reserved

The Abstract Data Type Queue

```
bool dequeue(QueueItemType& queueFront);

// Retrieves and deletes the front of a queue

// Precondition: None.
// Postcondition: If the queue is not empty,
// queueFront contains the item that was
// added to the queue earliest, and the item
// is deleted.
```

© 2005 Pearson Addison-Wesley. All rights reserved

The Abstract Data Type Queue

```
bool getFront(QueueItemType& queueFront)const;

// Retrieves the item at the front of a queue

// Precondition: None.
// Postcondition: If the queue is not empty,
// queueFront contains the item that was
// added to the queue earliest.
```

© 2005 Pearson Addison-Wesley. All rights reserved

Implementations of the ADT Queue

- An array-based implementation
- Possible implementations of a pointer-based queue
  - A linear linked list with two external references
    - A reference to the front
    - A reference to the back
  - A circular linked list with one external reference
    - A reference to the back

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

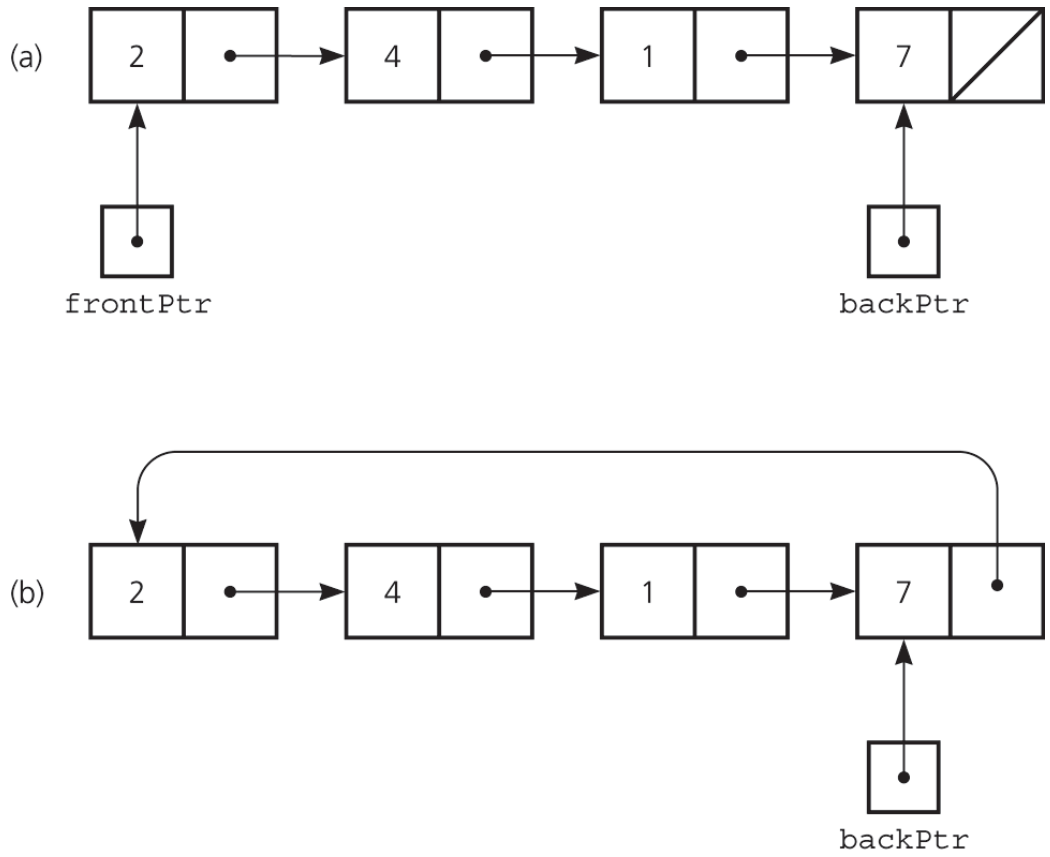


Figure 7.4 A pointer-based implementation of a queue: (a) a linear linked list with two external pointers b) a circular linear linked list with one external pointer

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

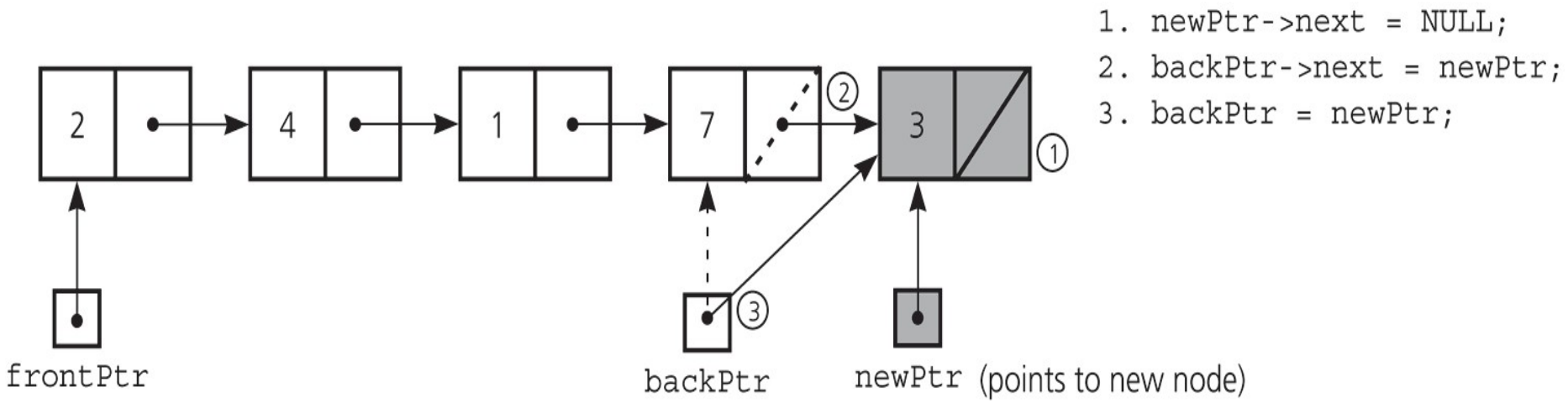


Figure 7.5 Inserting an item into a nonempty queue

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

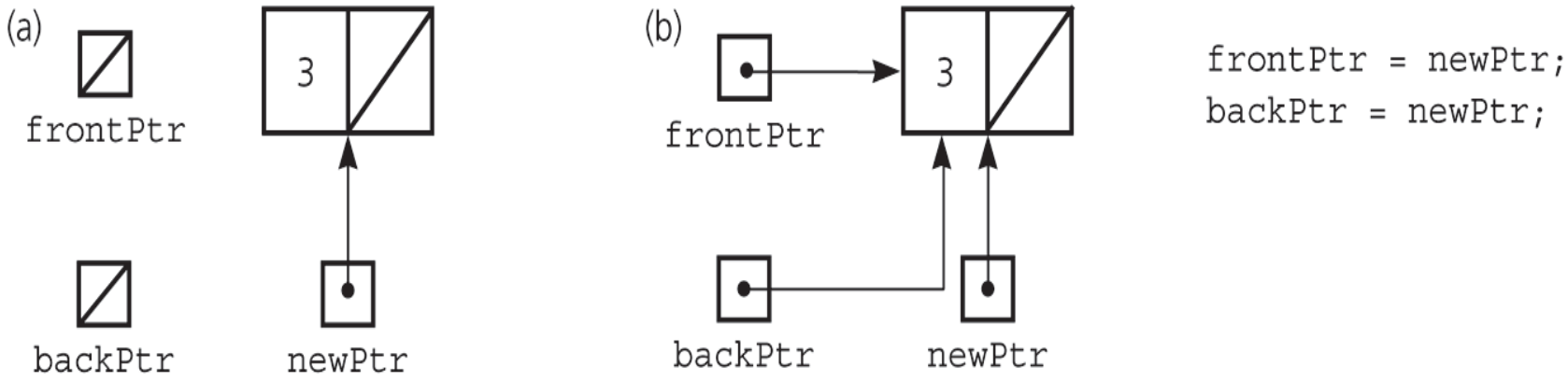


Figure 7.6 Inserting an item into an empty queue: a) before insertion; b) after insertion

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

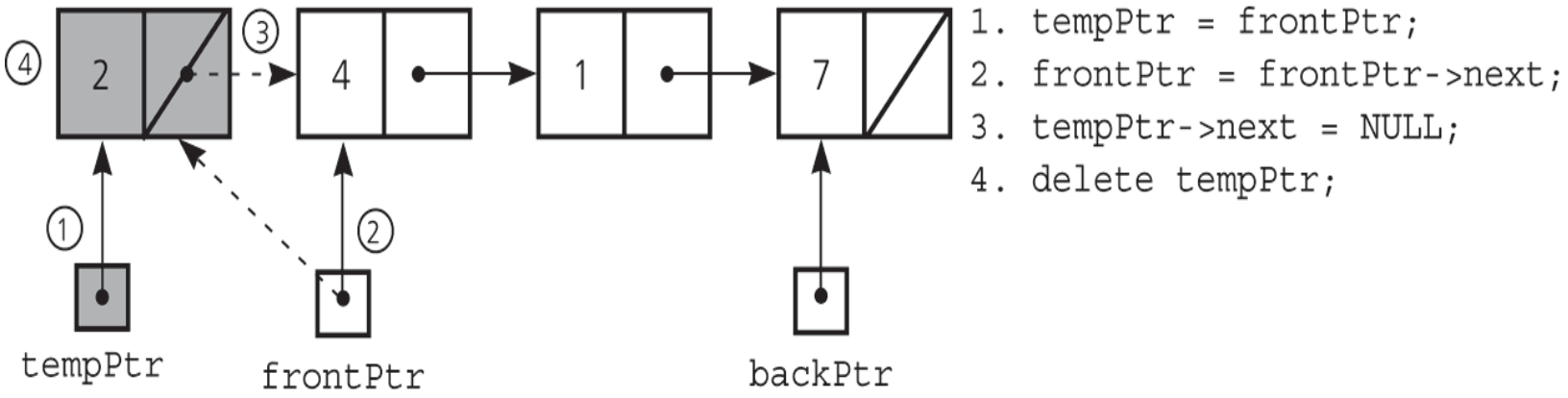


Figure 7.7 Deleting an item from a queue of more than one item

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

```
typedef desired-type-of-queue-item QueueItemType;

class Queue{
public:
    Queue();                          // default constructor
    Queue(const Queue& Q);            // copy constructor
    ~Queue();                         // destructor

    // Queue operations:
    bool isEmpty() const;
    bool enqueue(QueueItemType newItem);
    bool dequeue();
    bool dequeue(QueueItemType& queueFront);
    bool getFront(QueueItemType& queueFront) const;

private:
    // The queue is implemented as a linked list with one external
    // pointer to the front of the queue and a second external
    // pointer to the back of the queue.

    struct QueueNode{
        QueueItemType  item;
        QueueNode     *next;
    };
    QueueNode *backPtr;
    QueueNode *frontPtr;
};
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

```
// default constructor
Queue::Queue() : backPtr(NULL), frontPtr(NULL){

}
```

```
// copy constructor
Queue::Queue(const Queue& Q){

    // Implementation left as an exercise
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

```
// destructor
Queue::~Queue(){

     while (!isEmpty())
          dequeue();

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

```
bool Queue::isEmpty() const {
    return backPtr == NULL;
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

```
bool Queue::enqueue(QueueItemType newItem){

    // create a new node
    QueueNode *newPtr = new QueueNode;

    // set data portion of new node
    newPtr->item = newItem;
    newPtr->next = NULL;

    // insert the new node
    if (isEmpty())   // insertion into empty queue
        frontPtr = newPtr;
    else             // insertion into nonempty queue
        backPtr->next = newPtr;

    backPtr = newPtr;  // new node is at back

    return true;
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

```
bool Queue::dequeue(){

    if (isEmpty())
        return false;

    else{
        // queue is not empty; remove front
        QueueNode *tempPtr = frontPtr;

        if (frontPtr == backPtr){   // special case
            frontPtr = NULL;
            backPtr = NULL;
        }
        else
            frontPtr = frontPtr->next;

        tempPtr->next = NULL;  // defensive strategy
        delete tempPtr;
        return true;
    }
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

```
bool Queue::dequeue(QueueItemType& queueFront){

     if (isEmpty())
          return false;

     else{
          // queue is not empty; retrieve front
          queueFront = frontPtr->item;
          dequeue();  // delete front

          return true;
     }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

A Pointer-Based Implementation

```
bool Queue::getFront(QueueItemType& queueFront)const {

    if (isEmpty())
        return false;

    else {
        // queue is not empty; retrieve front
        queueFront = frontPtr->item;

        return true;
    }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation

```
const int MAX_QUEUE = maximum-size-of-queue;
typedef desired-type-of-queue-item QueueItemType;

class Queue{
public:
     Queue();  // default constructor
   // copy constructor and destructor are
   // supplied by the compiler

   // Queue operations:
   bool isEmpty() const;
   bool enqueue(QueueItemType newItem);
   bool dequeue();
   bool dequeue(QueueItemType& queueFront);
   bool getFront(QueueItemType& queueFront) const;

private:

   QueueItemType items[MAX_QUEUE];
   int           front;
   int           back;
   int           count;

};
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation

```
// default constructor
Queue::Queue():front(0), back(MAX_QUEUE-1), count(0){

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation

```
bool Queue::isEmpty() const {
    return (count == 0);
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation

```
bool Queue::enqueue(QueueItemType newItem) {

    if (count == MAX_QUEUE)
        return false;

    else{
        // queue is not full; insert item
        back = (back+1) % MAX_QUEUE;
        items[back] = newItem;
        ++count;

        return true;
    }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation

```
bool Queue::dequeue(){

     if (isEmpty())
          return false;

     else {
          // queue is not empty; remove front
          front = (front+1) % MAX_QUEUE;
          --count;

          return true;
     }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation

```
bool Queue::dequeue(QueueItemType& queueFront) {

    if (isEmpty())
        return false;

    else{
        // queue is not empty;
        // retrieve and remove front
        queueFront = items[front];
        front = (front+1) % MAX_QUEUE;
        --count;

        return true;
    }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Array-Based Implementation

```
bool Queue::getFront(QueueItemType& queueFront)const {

    if (isEmpty())
        return false;

    else{
        // queue is not empty; retrieve front
        queueFront = items[front];

        return true;
    }

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

- If the item in position 1 of a list `aList` represents the front of the queue, the following implementations can be used
  - `dequeue()`

    ```
    aList.remove(1)
    ```
  - `getFront(queueFront)`

    ```
    aList.retrieve(1, queueFront)
    ```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

- If the item at the end of the list represents the back of the queue, the following implementations can be used
  - `enqueue(newItem)`

```
aList.insert(getLength() + 1, newItem)
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
#include "ListP.h"  //  ADT list operations
typedef ListItemType QueueItemType;

class Queue {

public:
    Queue();                              // default constructor
    Queue(const Queue& Q);                // copy constructor
    ~Queue();                             // destructor

    // Queue operations:
    bool isEmpty() const;
    bool enqueue(QueueItemType newItem);
    bool dequeue();
    bool dequeue(QueueItemType& queueFront);
    bool getFront(QueueItemType& queueFront) const;

private:

    List aList;

};
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
// default constructor
Queue::Queue() {

}



// copy constructor
Queue::Queue(const Queue& Q): aList(Q.aList){

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
// destructor
Queue::~Queue() {

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Queue::isEmpty() const {
    return (aList.getLength() == 0);
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Queue::enqueue(QueueItemType newItem) {
    return aList.insert(aList.getLength()+1, newItem);
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Queue::dequeue() {
    return  aList.remove(1);
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Queue::dequeue(QueueItemType& queueFront) {

    if (aList.retrieve(1, queueFront))
        return aList.remove(1);
    else
        return false;

}
```

© 2005 Pearson Addison-Wesley. All rights reserved

An Implementation That Uses the ADT List

```
bool Queue::getFront(QueueItemType& queueFront) const{
    return aList.retrieve(1, queueFront);
}
```

© 2005 Pearson Addison-Wesley. All rights reserved

Example: Recognizing Palindromes

- A palindrome
  - A string of characters that reads the same from left to right as its does from right to left
- To recognize a palindrome, a queue can be used in conjunction with a stack
  - A stack reverses the order of occurrences
  - A queue preserves the order of occurrences

© 2005 Pearson Addison-Wesley. All rights reserved

Example: Recognizing Palindromes

- A nonrecursive recognition algorithm for palindromes
  - As you traverse the character string from left to right, insert each character into both a queue and a stack
  - Compare the characters at the front of the queue and the top of the stack

© 2005 Pearson Addison-Wesley. All rights reserved

Example: Recognizing Palindromes

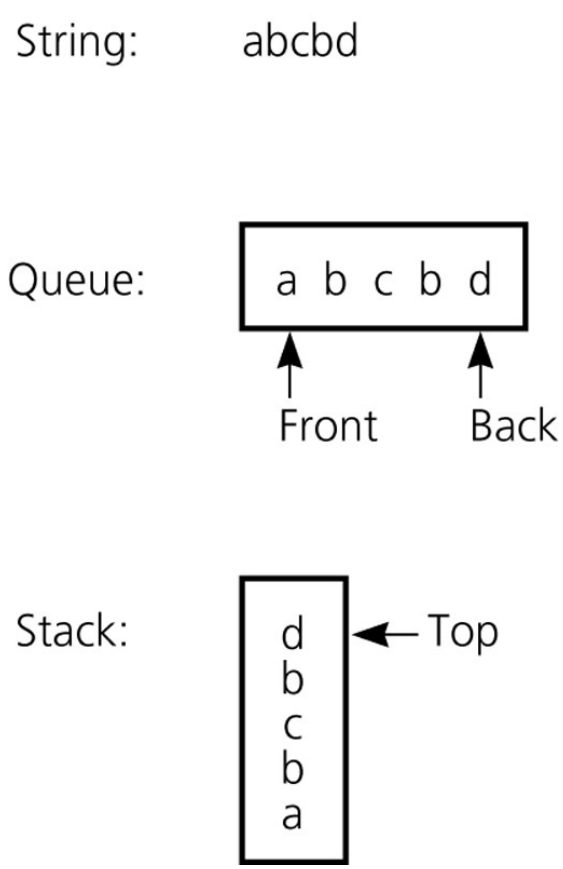


Figure 7.3

The results of inserting a string into both a queue and a stack

© 2005 Pearson Addison-Wesley. All rights reserved

Summary

- The definition of the queue operations gives the ADT queue first-in, first-out (FIFO) behavior
- A pointer-based implementation of a queue uses either
  - A circular linked list
  - A linear linked list with a head reference and a tail reference

© 2005 Pearson Addison-Wesley. All rights reserved

Summary

- A circular array eliminates the problem of rightward drift in an array-based implementation
- Distinguishing between the queue-full and queue-empty conditions in a circular array
  - Count the number of items in the queue
  - Use a `isFull` flag
  - Leave one array location empty

© 2005 Pearson Addison-Wesley. All rights reserved

Comparing Implementations

- Fixed size versus dynamic size
  - A statically allocated array
    - Prevents the `enqueue` operation from adding an item to the queue if the array is full
  - A resizable array or a reference-based implementation
    - Does not impose this restriction on the `enqueue` operation

© 2005 Pearson Addison-Wesley. All rights reserved

Comparing Implementations

- Pointer-based implementations
  - A linked list implementation
    - More efficient
  - The ADT list implementation
    - Simpler to write

© 2005 Pearson Addison-Wesley. All rights reserved

A Summary of Position-Oriented ADTs

- Position-oriented ADTs
  - List
  - Stack
  - Queue
- Stacks and queues
  - Only the end positions can be accessed
- Lists
  - All positions can be accessed

© 2005 Pearson Addison-Wesley. All rights reserved

A Summary of Position-Oriented ADTs

- Stacks and queues are very similar
  - Operations of stacks and queues can be paired off as
    - `createStack` and `createQueue`
    - Stack `isEmpty` and queue `isEmpty`
    - `push` and `enqueue`
    - `pop` and `dequeue`
    - Stack `getTop` and queue `getFront`

© 2005 Pearson Addison-Wesley. All rights reserved

A Summary of Position-Oriented ADTs

- ADT list operations generalize stack and queue operations
  - `getLength`
  - `insert`
  - `remove`
  - `retrieve`

© 2005 Pearson Addison-Wesley. All rights reserved